Two-wire power and signalling bus for modular SMA limb nodes

Architecture proposal · 20 V bus · 20 ms control frame · 1 cm node boards

This document sketches an architecture for a modular robotic limb in which each joint node drives two antagonistic muscle wires, and every node shares a single two-wire bus that carries both 21 V power and bidirectional signalling. The central idea is to split each 20 ms period into a short quiet signalling window and a longer power window, so data and muscle current never share the bus at the same time. Sections 9–12 add design refinements, alternative architectures, and a rated comparison of the options. Section 13 describes token mode, the default way to run the bus, in which control and power pass down the limb one node at a time. Section 14 sets out the layered protocol built on it, with bus mode as an optional upper layer, and section 15 describes the central controller board that drives up to six limbs.

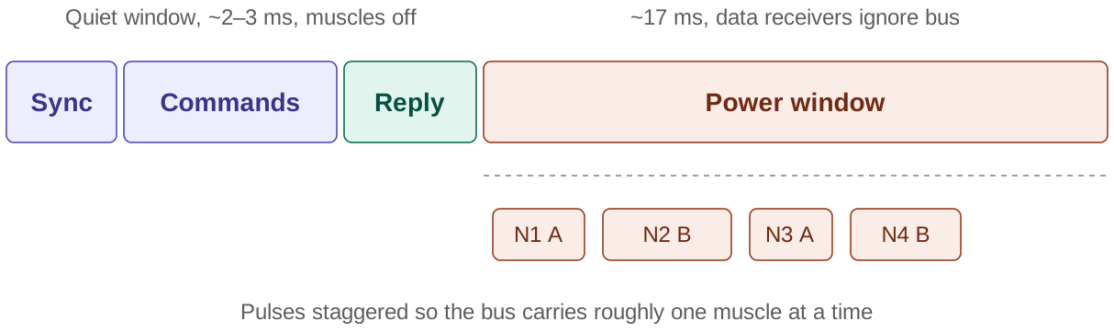


Figure 1. Structure of the 20 ms bus frame (not to scale).

1. Why time-division beats superimposing data on power

Muscle wires (assuming nitinol/SMA) are resistive loads drawing amps, so every pulse produces IR drop along the wires and a ground shift that varies along the chain. Any data scheme sharing the bus at the same moment has to fight that. If instead the first ~2–3 ms of every 20 ms frame is a quiet window (all muscles off), the bus is lightly loaded, clean, and predictable, and nodes simply ignore line voltage during the power window. That is the isolation, done in time rather than with transformers, which would not fit on a 1 cm board anyway.

A useful consequence: nodes do not need the on-time signalled in real time. The controller sends each node a duty value, the sync break gives every node a shared time reference, and each node runs its own timer for the rest of the frame.

2. Physical layer, downstream (controller → nodes)

Switch the bus between two levels: 21 V for idle or “1”, and 0 V for “0” and for the frame-start break, which is simply longer than any byte. An earlier version used about 12 V for “0” so that node logic stayed powered during data. The hold-up capacitor behind each node’s Schottky diode now does that job (section 13.3), so each limb port on the controller needs only a plain push-pull driver (section 15).

On the node, power comes in through a Schottky diode into a small hold-up capacitor and a 30 V-rated LDO (something in the TI TPS709 class) to 3.3 V. During 0 V bits the diode isolates the capacitor, which keeps the logic running; a byte of zeros (36 µs) uses a tiny fraction of its charge. A resistor divider (120 kΩ and 22 kΩ) brings 21 V down to about 3.25 V at the MCU’s UART receive pin. The pin reads high above about 15 V on the bus and low below about 6.5 V, so the margins are wide.

For encoding, borrow from LIN and DMX512: a long low “break” marks frame start, then a 0x55 sync byte lets cheap MCUs with ±2% RC oscillators autobaud, then standard UART bytes at around 250 kbaud. That gives roughly 40 µs per byte, so 16 nodes at 3 bytes each plus header and CRC fits within about 2 ms.

3. Physical layer, upstream (back-channel)

Nodes cannot easily pull a 20 V bus around, but they can modulate current. In a reply slot the node switches a resistor or small current sink to draw about 10–20 mA pulses, and the controller sees them across a shunt in the bus return with a comparator. This is the same idea as automotive PSI5 airbag sensors and HART-style current signalling. Since it happens in the quiet window, the only background current is the nodes’ logic, a few mA total, so the margin is easy.

Section 9.1 shows that the muscle current itself can replace this dedicated back-channel for presence, discovery and integrity. The reply slot then becomes optional, kept only if rich status (temperature, firmware version, fault codes) is wanted.

4. Chain-based addressing and discovery

Sections 4 and 5 describe bus mode, the first scheme considered. Section 13 reuses this chain-by-chain discovery as token mode, now the default, and section 14 keeps bus mode as an optional layer above it.

This is where the daisy chain earns its keep. Put a P-channel MOSFET in series with the + line on each node, upstream side to source, so its body diode blocks downstream when off. It defaults to open at power-up.

Enumeration then becomes:

1. At power-up only node 1 sees the bus. The controller sends ASSIGN 1.
2. Node 1 replies with a unique ID, module type, firmware version and measured muscle resistances.
3. The controller sends CLOSE_PASS; node 1 turns on its FET and node 2 powers up.
4. Repeat until an ASSIGN goes unanswered: that marks the end of the limb.

The address therefore equals physical position, which is what kinematics needs. The pass FET also provides fault isolation, since the controller can open the segment after a node that reports a short. The cost is that the FET carries all downstream current, so use a low-RDS(on) part (10–30 mΩ in a 2×2 mm DFN). With 20 V on the bus, clamp the gate with a ~12 V zener unless the FET is rated ±25 V VGS. The obvious trade-off is that a dead node takes out everything after it, but that is equally true of the wiring in any daisy chain.

A neat alternative is a WS2812-style scheme: each node consumes the first slot of the packet and re-transmits the remainder downstream through its pass FET, so no addresses exist at all. It is elegant, but every node’s timing errors and latency accumulate, and the pass FET has to switch at bit rate while carrying current. Enumeration with a transparent bus is the recommended approach.

5. Frame protocol

- **Break + sync:** time reference for the frame.
- **Header:** frame counter and which node is polled for reply this frame.
- **Slots:** one per node, holding a signed duty value. The sign selects muscle A or B, which suits an antagonistic pair. Add an optional second field for co-contraction (stiffness).
- **Start offset:** set once at enumeration, so each node knows when its pulse starts in the power window.
- **CRC.**

- **Reply slot:** the polled node returns status, fault flags and muscle resistance. Round-robin means every node reports every N frames.

6. Power: the numbers make a strong case for staggering

A worked example: 0.1 mm Flexinol is about 150 Ω/m with roughly 180 mA recommended continuous. A 10 cm muscle is about 15 Ω, so at 20 V it draws about 1.3 A peak (27 W). To match the recommended average heating (~0.5 W), duty is only about 2%, around 0.4 ms of the 20 ms frame.

The bus now runs at 21 V, so the same muscle draws about 1.4 A and heats about 10% faster for a given on-time. The controller measures the bus voltage and scales on-times to keep heating constant (section 15.5).

Two implications follow:

- **Resolution:** 8-bit duty is far too coarse in the region actually used. Send duty in timer ticks (e.g. 1 µs, 15 bits) and have each node enforce a maximum on-time, computed from the muscle resistance it measured, so a corrupted command cannot cook a wire.
- **Staggering:** pulses this short mean ~17 ms of power window can fit dozens of them without any overlap. If the controller schedules them back to back, the bus only ever carries about one muscle's current. Thin bus wires and small pass FETs become viable, and IR drop stops scaling with limb length.

7. Integrity comes almost for free

With non-overlapping pulses, the controller knows exactly which muscle is on at any instant, so its own current shunt measures that muscle's resistance. SMA resistance changes by roughly 10–20% through its phase transformation, giving a crude contraction estimate per muscle with no sensor on the node at all. It also detects open wires (no current), shorts (overcurrent) and missing nodes (no reply), and a node-side shunt can confirm it locally.

Make each node's firmware fail safe: if no valid frame with a good CRC arrives for about three frames, both muscles go off. Optionally interlock A and B so they cannot both be on unless co-contraction is explicitly commanded.

8. Fitting it in 1 cm

The board is a strip about 9 mm wide with the bus pads at each end:

- **MCU:** a small part with UART, timer, comparator and ADC, such as an ATtiny202-class or CH32V003-class chip in an 8-pin package or QFN.
- **LDO:** 30 V-rated, SOT-23-5 or DFN.
- **Switches:** dual N-MOSFET in DFN 2×2 for the two muscles (low-side, muscle from bus+ to drain); one P-MOSFET for pass-through plus a small N-FET or transistor to drive its gate.
- **Protection:** Schottky diode, hold-up capacitor of about 10 µF, a 24 V TVS across the bus, and divider resistors.
- **Back-channel:** a resistor and small transistor as the current sink.

This is all in the same density class as model-railway DCC decoders, which are routinely built around 5×8 mm while powering motors from a two-wire track carrying data. DCC is worth studying as a reference design. It uses polarity-reversing square waves and a bridge rectifier, which is very robust; section 10.4 assesses it as an alternative.

9. Design refinements

9.1 Muscle current as the back-channel

Because the controller schedules every pulse, it knows which muscle should be drawing current at each moment, and its own bus current shunt reads the result:

- **Current where expected:** the node is alive, its FET switched and the wire is intact. The current level gives the muscle's resistance, and from that a rough contraction estimate.
- **No current where expected:** open wire, dead FET or missing node.
- **Current where none was scheduled:** a stuck-on FET. The controller isolates it by opening the pass FET of the node upstream.

A muscle commanded to zero duty reveals nothing, so zero-duty muscles are polled once per epoch. With an epoch of, say, 50 frames (1 s), each idle muscle receives one 10 µs probe pulse per second. On the 15 Ω example that is about 0.27 mJ per probe, or roughly 0.3 mW of average heating: negligible against the ~0.5 W working budget. Discovery uses the same mechanism: after each CLOSE_PASS during enumeration, the controller commands a probe from the newest node and watches for current.

This removes the current-sink transistor and resistor from every node, and works identically for every option in section 10, including those without a microcontroller.

9.2 Fail-safe gate drive (capacitor-coupled)

Each muscle FET is driven through a series capacitor, with a resistor from gate to source and a clamp diode from source to gate. A rising edge from the drive logic couples through the capacitor and turns the FET on; the gate then decays through the resistor, so the FET can only stay on for a bounded time after each edge. A drive that sticks high, whether from a hung microcontroller, latched logic or a controller holding the bus, times out by itself. On the falling edge the clamp diode restores the capacitor's charge and stops the gate swinging negative.

This makes a safe maximum on-time a property of the hardware rather than of firmware or the controller, and it costs three passives per muscle. It belongs on every option. It applies only to the muscle FETs: the pass FET must stay on continuously and keeps a conventional DC gate drive.

Sizing. Maximum on-time is roughly $\tau \cdot \ln(V_{drive} / V_{th})$, with $\tau = C_c \cdot R_{gs}$. Keep Cc at least ten times the FET's input capacitance so the capacitive divider does not eat the drive. Example: Cc = 100 nF and Rgs = 15 kΩ give τ = 1.5 ms; with a 3.3 V drive and a 1 V threshold the FET switches off after about 1.8 ms. Choose the limit with margin above the longest legitimate pulse and well below the time that would overheat the wire.

Caveat. A slowly decaying gate passes the FET through its linear region, where it dissipates heavily at amp-level currents. Either feed the RC into a Schmitt buffer (such as a 74LVC1G17) that drives the gate, so turn-off is sharp, or accept a brief lossy turn-off that happens only under fault conditions.

9.3 Programming and field updates (microcontroller options)

Initial programming. Suitable parts have single-wire debug interfaces (UPDI on newer ATtiny parts, SWIO on the CH32V003). Two or three pads per board and a pogo-pin jig allow programming in the panel before depanelling.

Updating over the bus. Firmware update is the bootloader's first job, and it runs in token mode by default (section 14.2): the node holding the token is the only one listening, so the controller streams the image to it without needing an address. At 250 kbaud a 16 kB image takes about 0.66 s. A broadcast download to all nodes at once is added later as a bus-mode feature (section 14.4).

The rule that matters: the bootloader itself must be able to take part in enumeration and close the pass FET, or a half-flashed node cuts off everything downstream. Keep the bootloader in write-protected flash

that never erases itself, and have it check the application CRC at boot, staying in the bootloader if the check fails. This favours 8–16 kB parts over the smallest 2 kB devices.

10. Alternative architectures

10.1 Micro-less bucket-brigade timing

A distributed version of the decade-counter demultiplexer used in early RC receivers. There are no addresses, no firmware and no data bytes: the muscle on-times are simply the gaps between short dips on the bus, and the pass FET hands the bus from node to node.

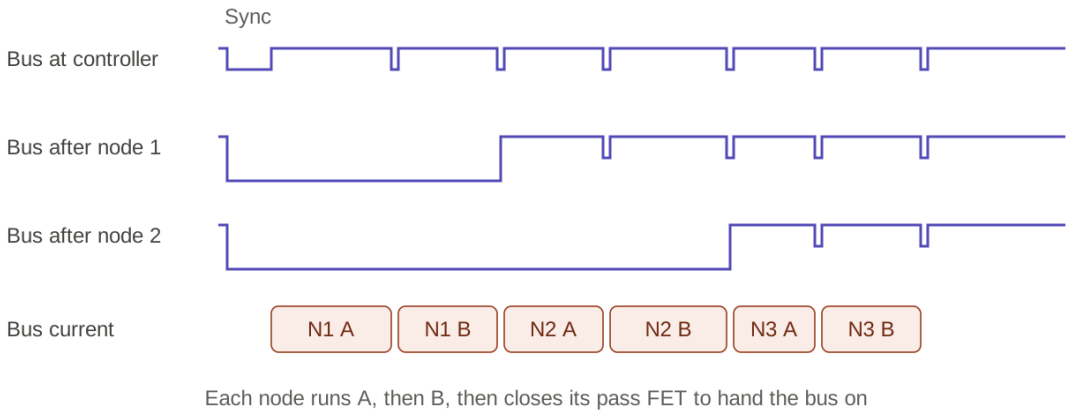


Figure 2. Bucket-brigade timing: bus segments and muscle current along the chain.

1. A long dip (the sync) resets every node, and each node opens its pass FET. Only node 1 still sees the bus.
2. When the bus returns high, node 1 turns muscle A on.
3. At the next short dip it switches to muscle B.
4. At the following dip it turns B off and closes its pass FET. Node 2's segment rises, which is node 2's start, and the pattern repeats down the chain.

Each node counts only two dips. The logic is a divider into a Schmitt input to detect dips, an RC filter so a long dip reads as reset and a short one as a clock, and a dual D flip-flop (such as a 74LVC2G74) as a 2-bit state machine driving the two muscle FETs and the pass FET. Nodes waiting their turn sit unpowered for up to ~17 ms, but CMOS logic drawing tens of µA from a 10 µF hold-up capacitor sags less than 0.1 V in that time.

Gains: no firmware; positional addressing for free; pulses are inherently sequential, so the bus never carries more than one muscle's current; presence and integrity checking come from the controller's shunt (section 9.1).

Costs: no co-contraction, since only one muscle is ever on; all on-times in the chain must fit in ~17 ms, which at ~0.4 ms per muscle is around 20 nodes, fewer with thicker wire or higher duty; a zero-duty muscle still costs a few µs of slot; a glitch scrambles the rest of that frame, though everything resynchronises at the next sync. Without a local controller, the capacitor-coupled gate timeout of section 9.2 is mandatory.

10.2 Bucket brigade in a configurable mixed-signal chip

Three small logic chips plus passives are not actually smaller or cheaper than a low-cost microcontroller. A configurable mixed-signal device such as a Renesas GreenPAK (around 2×3 mm) holds the dip comparator, sync filter, state machine and safety timeout in one chip, configured in a design tool rather

than programmed. There is no firmware to crash or update; changes mean reconfiguring parts on the bench (multi-time-programmable variants) or replacing them (one-time-programmable variants).

A microcontroller node can also implement the bucket-brigade timing as its baseline behaviour while accepting extra commands, so a system could start micro-less and add intelligence later without changing the bus.

10.3 DC power with superimposed differential AC signalling

Here the two wires carry a steady 20 V DC for power, and the signalling is a carrier (somewhere from a few hundred kHz to a few MHz, using on-off keying, FSK or a Manchester-coded baseband) superimposed differentially between the same two wires. The voltage-dip scheme is effectively the baseband, time-gated relative of this.

- **Node receive:** the carrier is coupled in through a small capacitor to a comparator. Every DC load on the node (the muscle feed and the logic supply) is separated from the bus by a series inductor or ferrite, so the low-impedance muscle and decoupling capacitors do not short the carrier.
- **Back-channel:** load modulation, as in RFID. The node switches a small RC across its carrier tap and the controller sees the amplitude change.
- **Controller:** drives the carrier through a small transformer or a class-D stage, and feeds the DC through an inductor of its own.

Gains: signalling is continuous and independent of the power window, so there is no quiet window, muscle pulses may overlap, and bandwidth can be much higher (useful for high-rate closed-loop sensing at the nodes). Chain enumeration through the pass FETs works unchanged, since the closed pass FETs carry DC and carrier alike.

Costs: the muscle-feed inductor must carry peak muscle current (around 1.5 A), making it one of the largest parts on a 1 cm board. Each node's parasitics attenuate the carrier, limiting chain length or demanding gain control. Muscle switching edges are broadband noise directly in the signalling band, so muscle FETs need slew-rate limiting and the coding needs error detection. The controller's analogue front end is more involved.

10.4 DCC-style bipolar bus

As in model-railway DCC, the controller drives a polarity-reversing square wave; each node rectifies it for power and decodes data from the timing of the reversals.

Gains: connectors can be fitted either way round; decades of field experience; existing commercial decoders demonstrate the density.

Costs: the bridge rectifier drops roughly 0.8–1.5 V at peak muscle current. At ~2% duty the average dissipation is only tens of mW, so the real penalties are reduced muscle voltage and the diodes' peak-current rating. The controller needs an H-bridge switching full bus current, and polarity reversals during a muscle pulse need a hold-up capacitor behind the bridge or reversals confined to the quiet window. The pass FET must now block both polarities, which needs two back-to-back FETs per node.

11. Comparison of options

Each option is scored 1–5 on each criterion, where 5 is best: for dollars, 5 means cheapest; for board size, 5 means smallest. The total is unweighted and only a starting point, since the weighting depends on the application. Estimated costs are rough per-node bill-of-materials figures at around 1,000 units, excluding the PCB and assembly, and will vary with supplier and component choice. All options are assumed to use the capacitor-coupled gate timeout and muscle-current presence detection from section 9.

Option	Dollars	Size	Expand	Reliab.	Adapt.	Total	Est. BOM / node
A. MCU, voltage-dip	4	4	5	4	5	**22**	$0.90–1.40
B. Discrete-logic brigade	4	3	2	4	1	**14**	$0.90–1.30
C. Mixed-signal brigade	4	5	2	4	2	**17**	$0.80–1.20
D. DC + AC carrier	2	2	4	3	5	**16**	$1.40–2.20
E. DCC-style bipolar	3	3	4	4	5	**19**	$1.30–2.00

What drives each rating

- **A. Microcontroller, voltage-dip signalling:** a sub-dollar MCU in an 8-pin package is about as cheap and small as any logic solution. Highest adaptability (firmware updates over the bus) and expandability (addresses, overlapping pulses, optional co-contraction and status). Reliability is good but depends on a sound bootloader design.
- **B. Discrete-logic bucket brigade:** no firmware to fail, but three or more small chips take more area than one MCU. Expandability is capped by the shared 17 ms time budget, and behaviour can only change by redesigning the board.
- **C. Mixed-signal-chip bucket brigade:** the smallest board, no firmware, and modest cost. Same time-budget limit as B; reconfigurable only on the bench.
- **D. DC with superimposed AC carrier:** the most capable communications channel, but the high-current inductor and analogue front end cost money and area, and switching noise in the signalling band makes reliability harder to achieve.
- **E. DCC-style bipolar bus:** wiring-polarity-proof and well proven. The bridge, back-to-back pass FETs and a full-current H-bridge at the controller add cost and area.

12. Recommendation

- **Default:** option A, with muscle-current presence detection, epoch probing of idle muscles, and capacitor-coupled gate timeouts. It leads on adaptability and expandability at close to minimum cost and size.
- **If firmware is unwanted, or for a first prototype:** option C. If option A nodes implement the bucket-brigade timing as their baseline mode, the two can be mixed on one bus and the system can migrate later.
- **If the nodes must communicate during the power window:** option D, accepting the larger board.
- **If reversed wiring is a real field risk:** option E.

- **Operating mode:** token mode is the default, from discovery and firmware update through to muscle activation (section 13). Bus mode is added later as an optional layer (section 14). Both run on the same option A hardware.

13. Token mode (default)

Token mode is the default way to run the bus. It is used from power-up for discovery, enumeration and firmware updates, and then for muscle activation. At the start of every frame the pass switches reopen, and control and power ripple down the limb one node at a time. A node listens only while its own pass switch is open, so only one node is ever listening. For muscle activation its whole message is the on-time for muscle A and the on-time for muscle B.

Bus mode, in which every pass switch stays closed and every node hears every frame, becomes an optional upper layer that relies on addresses handed out in token mode (section 14).

13.1 How a frame runs

1. **Break.** The controller drops the bus to 0 V for about 100 µs. Every node, all of them powered at the end of the previous frame, opens its pass switch and goes to sleep. Only node 1 still has bus power.
2. **Wake.** Node 1 wakes when its segment's voltage returns and acknowledges with a 10 µs probe pulse on one muscle, which the controller sees on its current sensor. The controller then waits about 50 µs in total for the node's clock to settle.
3. **Message.** The controller sends node 1 its message: a type byte, the payload and a check byte. For muscle activation the payload is the two on-times; other types carry discovery, enumeration and firmware data.
4. **Run.** Node 1 runs muscle A, then muscle B, then closes its pass switch. Segment 2 powers up and node 2 wakes and acknowledges.
5. **Repeat.** The controller sends node 2's message, and so on down the limb.
6. **Broadcast.** After the last node, every node is powered and listening, so an optional broadcast slot can carry commands for all of them, such as "all stop", "start applications" or "switch to bus mode".

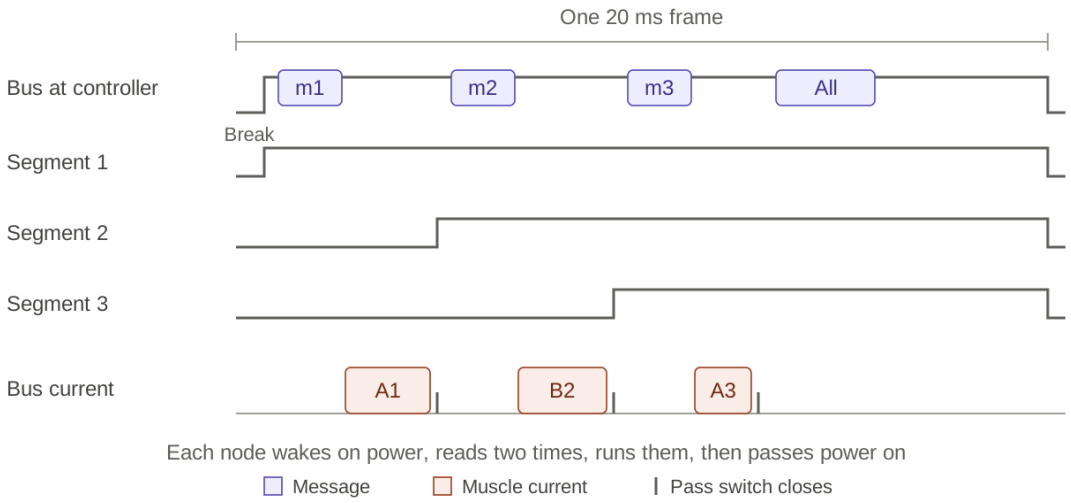


Figure 3. Token mode: each segment powers up only after the node before it has finished its turn.

This is the bucket-brigade timing of section 10.1 carried out by microcontroller nodes, with the on-times sent as data rather than as gaps between dips.

13.2 Why it suits this system

- **No addresses needed and almost no parsing.** A node's place in the chain identifies it, and a muscle message is two numbers and a check byte, so the node firmware is very small.
- **Data and muscle current never overlap.** While a node runs its pulses, nothing downstream is powered and nothing is being sent. This gives the separation that bus mode needs a quiet window for.
- **One muscle at a time.** Pulses are sequential by construction, so peak bus current never exceeds one muscle and thin bus wire is enough.
- **Every node is confirmed every frame.** The controller sees each node's wake acknowledgment and its muscle pulses on the current sensor, so a missing or faulty node is identified by position within one frame. Discovery is simply counting acknowledgments until none arrives.
- **Faults are short-lived.** A corrupted message costs at most one frame, because the next break resets every node.
- **Pulse position does not matter.** Nodes further down fire later in the frame, but SMA responds thermally over tens to hundreds of milliseconds.

13.3 Power between turns

Nodes further down the limb lose bus power for part of every frame, up to about 17 ms for the last node. They do not reboot: the hold-up capacitor C1, behind D1, keeps the microcontroller alive. A 10 µF 35 V X5R capacitor in 0805 retains roughly 3 µF at 20 V, and between about 19.6 V and the LDO's 3.6 V limit that gives about 48 µC of usable charge. While the segment is unpowered nothing else draws on it: D1 blocks reverse current and the data divider is fed from the bus, not from C1.

Node state while its segment is unpowered	Hold-up lasts
Asleep, waking on the supply returning (about 5 µA including the LDO)	About 10 s
Awake at a slow clock (about 0.5 mA)	About 96 ms
Awake at full speed (about 3 mA)	About 16 ms: marginal

The firmware rule is therefore to **sleep while waiting**. With the node asleep, a 17 ms gap uses well under 1% of the stored charge. No keep-alive pulses are needed.

13.4 Frame time budget

Each node costs a fixed overhead of about 0.23 ms: a 4-byte message at 250 kbaud (0.16 ms), about 0.05 ms to wake, and about 0.02 ms to hand over. The rest of the frame is the sum of all commanded on-times. Antagonistic pairs rarely co-contract, so A plus B is usually close to one muscle's on-time, and a muscle commanded to zero costs nothing beyond its occasional 10 µs probe.

On-time per node (one muscle)	Nodes that fit in 19 ms
0.4 ms (the worked example in section 6)	About 30
1.0 ms	About 15
1.8 ms (the hardware safety limit)	About 9

If a long limb ever needs more, the controller can cap each node's on-time per frame and spread the rest over the next frame. Because SMA heating is slow, this has little visible effect.

13.5 Safety rules

- **Keep the capacitor-coupled gate timeouts** (section 9.2). They bound every pulse even if node firmware hangs.
- **No valid message, no pass.** If a node's message does not arrive within about 1 ms of waking, or fails its check byte, the node leaves its muscles off and its pass switch open. The chain stops at a fault instead of passing bad data downstream.
- **The break always wins.** A break opens every pass switch and returns every node to sleep, whatever state it was in.

13.6 Bus mode and token mode compared

	Bus mode	Token mode
Who listens	Every node, every frame	Only the node whose turn it is (all, in the broadcast slot)
Addressing	Unique address, assigned during token-mode enumeration	None needed: position is implicit
Node message	Its slot in a frame for all nodes	Two on-times and a check byte
Data and muscle current	Separated by the quiet window	Never overlap, by construction
Peak bus current	Set by the controller; more than one muscle when activation overlaps	One muscle, always
Frame time limit	Quiet window plus all pulses	All on-times plus about 0.23 ms per node
Node power	Continuous; nodes sleep between messages	Downstream nodes sleep on C1 for part of each frame
Confirmation	Muscle current and addressed replies	Wake acknowledgment and muscle current, every frame
Role	Optional layer: broadcast download, overlapping activation, continuous power	Default: discovery, enumeration, firmware update, muscle activation

Both modes run on the same node hardware. Token mode is always available from the bootloader; bus mode is an application layer that is entered by a broadcast command and left by a long break (section 14).

14. Layered protocol

The protocol is built in layers, each relying only on the ones below it. The lowest layer lives in the write-protected bootloader, so a node can always be found, numbered and updated, whatever state its application is in. New features are added as upper layers as ideas are tried, without changing the layers beneath.

Layer	Lives in	Provides
0. Token core	Bootloader (write-protected)	Break, wake and acknowledgment; pass-switch control; discovery; enumeration; firmware update
1. Muscle activation	Application	Two on-times per node in token mode, with the safety rules of section 13.5
2. Bus mode	Application	Addressed messages; nodes that sleep and wake for each message; continuous power
3. Bus features	Application	Broadcast download, overlapping muscle activation, and further features as they are needed

14.1 Start-up sequence

1. **Power-up.** Every pass switch is open, so only node 1 is powered. Every node starts in its bootloader.
2. **Discovery.** Token passes run down the limb. Each wake acknowledgment counts one node, until a pass gets no answer.
3. **Enumeration.** In the same pass, the controller gives each node an address: its position in the chain. The node keeps it for bus mode.
4. **Firmware check and update.** The controller sends each node the firmware version it expects. The node answers yes or no with one or two probe pulses, and any node that answers no is updated before going further.
5. **Run.** A broadcast starts every node’s application, and the limb runs in token mode with muscle activation.
6. **Bus mode, when wanted.** A broadcast switches the limb to bus mode. A long break, several milliseconds at 0 V, always returns every node to token mode in its bootloader.

14.2 Firmware updates in token mode

The node holding the token is the only one listening, so it needs no address: the controller keeps the token with that node and streams the image to it. A 16 kB image takes about 0.66 s at 250 kbaud, plus flash write time, so roughly 1–2 s per node. The node checks the image’s CRC and acknowledges, and the controller confirms before starting the new firmware.

While one node is updated, the nodes below it sleep on their hold-up capacitors, which last about 10 s. If an update ever takes longer, those nodes simply start from cold when power returns, which the bootloader handles in any case.

Because this path lives entirely in the bootloader, it always works, even if an application is faulty. That makes it the safe default; the faster broadcast download of section 14.4 is an addition, not a replacement.

14.3 Bus mode with sleeping nodes

After enumeration every node has a unique address, so bus mode can add addressed messages. Pass switches stay closed, and each node repeats a simple cycle:

1. Sleep in a low-power state, with only its pass switch held on.
2. Wake on the start of a message.
3. Read the header and address.
4. Act if it is the recipient, or if the message is a broadcast.
5. Return to sleep.

The ATtiny's UART can wake the chip from standby on a start bit, but the first byte may be lost while the clock starts, so every message begins with a break and a sync byte (section 2).

A sleeping node draws very little itself. The main standing cost is holding its pass switch on, which in the schematic's current values is set by R4:

Standby draw per node from the 21 V bus	Current (power)
Pass switch held on, R4 = 10 kΩ (current schematic)	About 1.2 mA (25 mW)
Pass switch held on, R4 = 100 kΩ (recommended)	About 0.12 mA (2.5 mW)
Data receiver (R1, R2)	About 0.15 mA
Microcontroller asleep, plus the LDO	A few µA

Recommendation: raise R4 to 100 kΩ. A ten-node limb then idles at about 2.7 mA (about 56 mW) instead of about 13 mA. The cost is speed: Q3 turns on in about 65 µs instead of about 6 µs, and turns off through R3 in about 75 µs either way. In token mode this adds about 0.05 ms per hand-over, which the frame budget of section 13.4 easily absorbs.

14.4 Bus-mode features

- **Broadcast download.** Every node of one type receives a firmware image at the same time, so the whole limb is updated in about the time of one image. A token pass then checks each node's CRC with a yes-or-no acknowledgment.
- **Overlapping muscle activation.** Several muscles can be on at once, for co-contraction (stiffness), more total on-time per frame, or faster response. Peak bus current rises with the number on at once, so the controller caps it to what the wire gauge and pass switches carry; each pass switch carries the current of every node below it. Data stays in a quiet window, and each node times its pulses from the frame sync, as in section 5.
- **Continuous power.** Every node stays powered, so nodes can run sensors or local control loops, and no node depends on its hold-up capacitor.
- **Always recoverable.** However an upper layer behaves, a long break returns every node to token mode in its bootloader, so experiments in the upper layers can always be recovered from.

15. Central controller

One board talks to every limb. It carries an Arduino Nano R4 soldered to pads on the underside, six identical limb ports (fit four or six), a four-pin header that carries the 21 V input and a GND/TX/RX console, and a 6-DOF IMU on the Nano's own 3.3 V I2C bus. The KiCad project limb_controller (schematic rev 0.5, placement rev 0.3) holds the schematic and a placed board.

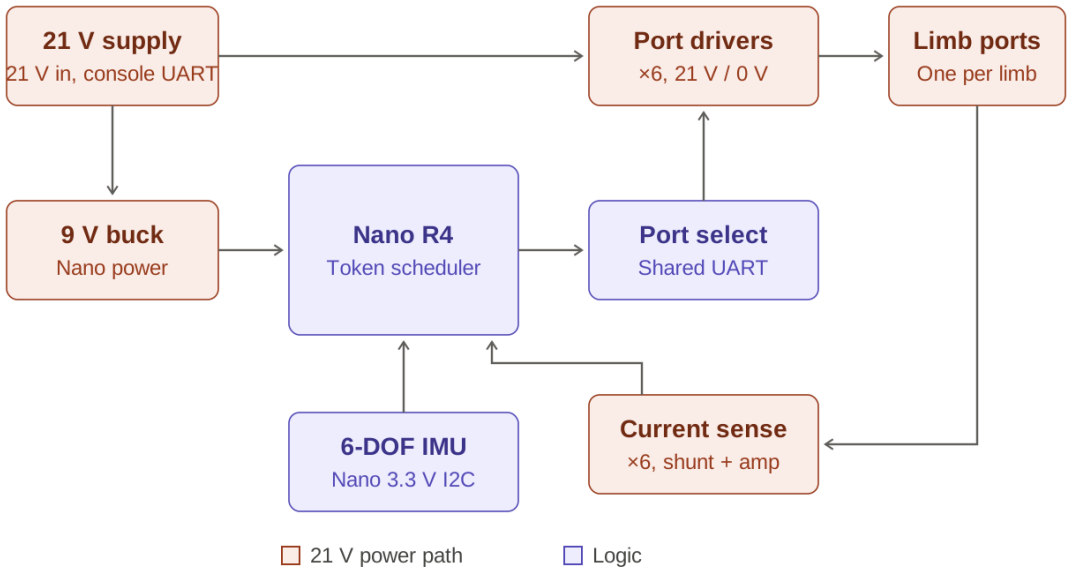


Figure 4. Central controller: power path and logic.

15.1 One UART for every limb

One UART serves every limb: SCI9 on D11 (transmit) and D12 (receive). The Nano's other UART, on D0/D1, is the console. One limb UART is enough. In token mode a node's message takes about 0.16 ms while its muscle pulse takes 0.4–1.8 ms, so the UART is idle most of the time. The firmware interleaves the limbs: it sends limb 1's node its message, then limb 2's while limb 1's muscle runs, and so on. Six limbs of five nodes need only about 5 ms of UART time per 20 ms frame.

A 74HC138 decoder routes the UART to one port at a time: the UART output drives one of its enable inputs, so the selected port follows the data while every other port sits at 21 V. A 74HC595 shift register holds each port's power on or off, for powering a limb down, isolating a fault, or sending the frame-start break to every limb at once. One NAND gate per port combines the two. Pull-downs on the shift register outputs keep every port off until the Nano has loaded it.

15.2 Discovering limbs and segments

No configuration is needed for a four- or six-limb build. At start-up the Nano runs a token discovery pass on every port. A port that never acknowledges has no limb; on a port that does, each wake acknowledgment counts one segment, until a pass gets no answer. Fitting four limbs is simply a matter of which ports are populated.

15.3 Limb port circuit

- **High side:** an AO3401A P-channel FET switches the port to 21 V and carries the muscle current. A 2N7002 level shifter, clamped by a 10 V zener, drives a complementary transistor pair that charges and discharges the FET's gate quickly in both directions.

- **Low side:** a 2N7002 pulls the port to 0 V through 10 Ω. Its gate is driven through an RC delay with a bypass diode, so it turns on late and off early and the two FETs never conduct together.
- **Protection:** a 1.5 A resettable fuse, an SMF24A TVS across the connector, and a 2-pin JST-PH connector.
- **Current sense:** a 50 mΩ low-side shunt and an INA181A1 amplifier (1 V per A), wired straight to its own Nano analog input, with no mux. The Nano's ADC scans all six ports and the bus voltage in one pass, a few microseconds per channel. This detects wake acknowledgments, measures muscle resistance and spots faults.
- **Kelvin sense connection:** the amplifier reads the shunt through its own pair of sense tracks (SNSk+ and SNSk−), taken from small pads at the inner edges of the shunt's two lands and net-tied to them. Neither sense track touches the ground plane. Without this, the amplifier's minus input would sit on a ground via a few millimetres from the shunt. Ground-plane drop from other ports' muscle currents, the 9 V converter and the logic would then add roughly 5–30 mA of error, the same size as the resistance change used to track wire heating.

15.4 Nano pin assignment

Function	Nano R4 pins
Limb UART (SCI9)	D11 (TX, shared by all ports), D12 (RX)
Console UART (SCI2), on J1 pins 3 and 4	D1 (TX), D0 (RX)
Port select (74HC138)	D5, D3, D4
Port power (74HC595 data, clock, latch, enable)	D6, D7, D8, D9
Port current, ports 1–6 (no mux)	A0–A5
Bus voltage	A6
IMU interrupt	D2 (external interrupt IRQ0)
IMU I2C	Qwiic-side holes (3.3 V, second I2C bus)
Spare	D10, D13, A7

15.5 Power

- **Input and console:** J1 is a 1 × 4 header, the same as on the octagon board: pin 1 is 21 V, pin 2 GND, pin 3 TX (Nano D1) and pin 4 RX (Nano D0). Pins 2–4 form a GND/TX/RX console header, so never plug a UART cable onto pins 1–2.
- **Input protection:** a P-channel FET for reverse-polarity protection, a P-channel FET for reverse-polarity protection, an SMBJ24A TVS and 470 µF of bulk capacitance.
- **Nano supply:** a 9 V buck converter (AP63200) feeds the Nano's VIN. The Nano's VIN limit is 21 V, so it cannot take the raw supply with any margin. The logic chips run from the Nano's 5 V output and the IMU from its 3.3 V output.
- **Bus voltage compensation:** a divider lets the Nano read the bus voltage. Muscle heating scales with voltage squared, so the firmware scales on-times to keep heating constant as the supply varies.

- **Input current:** six limbs firing at once could peak near 9 A, but 2.54 mm header pins carry about 3 A each. Either the firmware limits how many muscles are on at once, or the input uses a heavier connector, such as 3.96 mm or 5.08 mm pitch.

15.6 IMU

The IMU borrows a trick from the octagon board: it uses the Nano's own 3.3 V I2C bus, the one behind the Qwiic connector. The Nano has plated through-holes for SCL and SDA beside that connector. The bus is level-shifted and pulled up to 3.3 V with 5.1 kΩ on the Nano. So the board needs no translator and no pull-ups, only two plated holes under the Nano's holes and a pin or wire through each. The pins also align the Nano for soldering. The IMU takes 3.3 V from the Nano's 3V3 pin. Its interrupt still needs a single-FET level shifter to reach a 5 V input. It goes to D2, an external-interrupt pin, as on the octagon board. The IMU (LSM6DSOX, address 0x6A) sits at the exact centre of the board, directly above the Nano.

Five more plated holes sit over the SWD test pads on the Nano's underside (SWDIO, SWCLK, reset, GND, 3V3). With the Nano soldered down, a pogo pin or a wire through each hole still reaches the debugger pads.

15.7 Board size and construction

Three choices shape the board: the Nano R4 is soldered to the underside, the board is 0.8 mm thick, and it is the smallest rectangle that holds the parts, since weight follows area. The estimate below, made before placement, adds each part's footprint and allows room for routing: about 2.5 times the part area for the ports and the IMU, and less for the simpler sections. With the Nano underneath, it takes no top-side area.

Section	Top-side area including routing
Six limb ports (about 20 × 23 mm each)	About 2,570 mm²
Port logic: 74HC138, 74HC595, three 74HC00	About 800 mm²
Power input	About 340 mm²
9 V buck converter	About 150 mm²
IMU (no level translation now)	About 60 mm²
Nano R4 (45 × 18 mm)	None: on the underside
Total (estimate)	About 3,920 mm²

Placing the real parts, with routing gaps, fits a smaller board than the estimate: **60 × 58 mm** (3,480 mm²), down from 72 × 72 mm. Dropping the analog mux saved 4 mm of width. It has three ports down each side with the connectors on the edges, and the power section along the top edge with J1. The port logic sits in the centre column with the IMU at the board centre. The Nano lies underneath, and its USB-C end overhangs the front edge by 2 mm, as on the octagon board. Every part stays at 0° or 90°.

A rectangle is the lightest shape here. An octagon only helps if the limb ports sit on the angled edges, and that means parts at 45°, which caused routing trouble before. With every part square to the board, the octagon's cut corners just remove space the ports need.

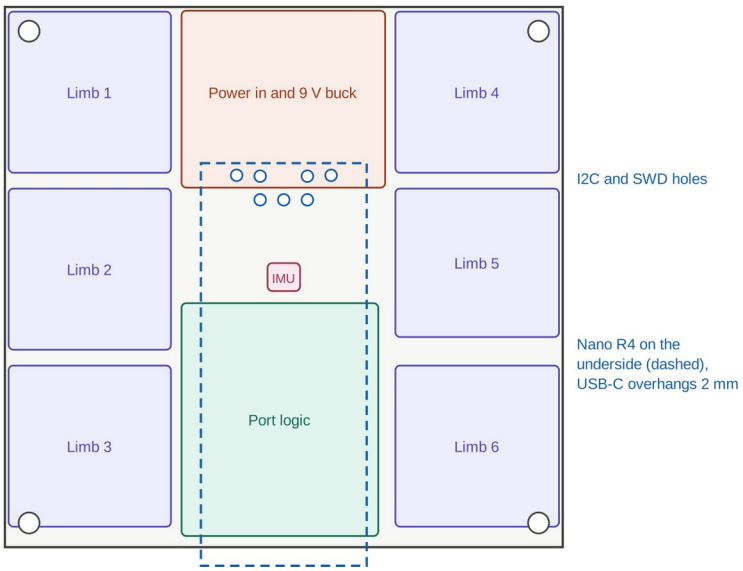


Figure 5. Central controller as placed, 60 × 58 mm, with the Nano R4 underneath.

With the Nano on the underside:

- **Footprint:** the Nano's castellated pads solder to its footprint placed on the bottom side. In KiCad, flip the footprint to the back rather than mirroring the pinout by hand. Its signals reach the top through vias. A keep-out under the Nano stops pads and parts on the back where its bare test pads face the board. It ends short of the I2C and SWD holes at the Qwiic end.
- **Assembly:** have the top side machine-assembled, then hand-solder the Nano. Castellated edges are easy to solder by hand, and the Nano avoids a second reflow upside down.
- **Access and clearance:** the Nano's USB-C connector, reset button and LEDs face down. Leave clearance under the board for the Nano and its connector, and a route for a USB cable during development. Check that the Nano R4's own underside is flat before relying on flat mounting.

With a 0.8 mm board:

- **Weight:** the bare 60 × 58 mm board weighs about 5.2 g, against 7.7 g at 72 × 72 mm and about 19 g for 80 × 80 mm at 1.6 mm. Moving the port parts to both sides would roughly halve the area again, at the cost of two-sided assembly.
- **Stiffness:** a 0.8 mm board is about one-eighth as stiff as a 1.6 mm one. If the board is part of the body structure or carries limb loads, support it at four or more mounting points, such as M2 holes near the corners and the centre.
- **Heavy parts:** the 470 µF electrolytic capacitor is the heaviest and tallest part, about 10.5 mm. On a thin board under vibration, several low-profile polymer or ceramic capacitors are a better choice.
- **Connectors:** through-hole JST-PH and header pins work in 0.8 mm boards, but their solder fillets are shorter. Check each connector's supported board thickness, and consider mechanical retention for the 21 V input.
- **Fabrication and current:** 0.8 mm is a standard thickness at common PCB makers. Current capacity depends on copper weight, not board thickness, so use copper pours for the 21 V and ground paths.

Open decisions

The main open choices are the muscle wire diameter and length, since those set peak current and how tightly the pulse schedule can be packed; whether co-contraction matters for the limb; the epoch length for polling idle muscles; the token-mode break, wake and message timings, which should be confirmed on real nodes; the R4 value, which trades pass-switch hold current against switching time; the two break lengths that mark a token frame and a return to the bootloader; the controller's input connector, which must carry the peak current of all limbs; and how the controller board is mounted, given the Nano underneath and the thinner board; what feeds the limb UART's receive pin (D12), since node replies are read as current today; and whether to adopt the octagon board's ICM-42688-P IMU so the existing posture firmware carries over.